

Examen 7 janvier 2019, durée 1h30 (15h15-16h45)

Documents et appareils électroniques prohibés

QCM (10 points)

Bonne réponse +1 point, aucune réponse 0 point et mauvaise réponse -0,5 point.

1. Quelle est la valeur maximale prise par `gridDim.x*threadIdx.x` lorsqu'on exécute `kernel<<<16, 128>>>();` ?
 - a) 2048
 - b) 2032
 - c) 1905
2. Le GPU favorise une implémentation avec
 - a) Une communication très réduite entre les threads
 - b) Une communication forte entre les threads
 - c) Entre a) et b) selon l'algorithme
3. Supposons un block de 64 threads exécutant la ligne `printf("%i, ", threadIdx.x);`
On obtient
 - a) Obligatoirement : 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63,
 - b) Un affichage très aléatoire, par exemple : 0, 1, 33, 54, ...
 - c) Ni l'un ni l'autre
4. Parmi les syntaxes suivantes, laquelle des trois transfère la valeur de 64 bits du CPU vers le GPU ?
 - a) `cudaMemcpy(A, a, 2*sizeof(float), cudaMemcpyHostToDevice);`
 - b) `cudaMemcpy(A, a, 2*sizeof(int), cudaMemcpyDeviceToHost);`
 - c) `cudaMemcpy(A, a, sizeof(float), cudaMemcpyHostToDevice);`
5. `__syncthreads();` permet de synchroniser
 - a) Chaque warp
 - b) Les threads
 - c) Les blocks
6. Pour certaines tâches, le GPU est plus efficace que le CPU car
 - a) C'est un coprocesseur
 - b) Sa mémoire a une plus grande bande passante
 - c) Ses unités de calcul sont plus rapides
7. Désigner la fausse information
 - a) On peut lancer plus de blocks que de threads par blocks
 - b) Le nombre de threads par block doit être nécessairement égal à une puissance de 2
 - c) Le nombre de threads par block doit être de préférence égal à une puissance de 2

8. `atomicAdd` est une fonction qui
 - a) Permet de séquentialiser une addition
 - b) Rend possible l'utilisation de la mémoire shared
 - c) Permet de paralléliser une addition
9. Quelle est la valeur maximale prise par `blockDim.x*blockIdx.x` lorsqu'on exécute `kernel<<<16, 128>>>();` ?
 - a) 2048
 - b) 1905
 - c) 1920
10. Le passage à l'échelle de la loi d'Amdahl en fonction du nombre de processeurs P est
 - a) Linéaire
 - b) Plus important que la loi de Gustafson lorsque $P = 2$
 - c) Asymptotiquement majoré

Problème : La plus grande valeur d'un tableau (10 points)

Utilisant `biggest_k<<<numBlocks, threadsPerBlock>>>(int *In, int *Out, int N)`, nous voulons trouver le plus grand élément `Out` d'un tableau `In` de N entiers positifs non triés. Pour simplifier davantage l'exercice, la mémoire shared est à allouer de façon statique, par exemple : `__shared__ int InSh[64];`

L'étudiant peut utiliser la même définition de kernel pour plusieurs questions à condition d'expliquer en quoi la définition est générique et sans impact négatif sur la rapidité du calcul.

1. Définir le kernel `biggest_k` utilisant le maximum de threads et permettant de trouver le maximum du tableau `In` lorsqu'on exécute
 - a) `biggest_k<<<1, 1>>>(In, Out, 32);` (1 point)
 - b) `biggest_k<<<1, 2>>>(In, Out, 32);` (1 point)
 - c) `biggest_k<<<1, 4>>>(In, Out, 32);` (1,5 point)
 - d) `biggest_k<<<1, 64>>>(In, Out, 128);` (2 points)
2. Contrairement à la définition du kernel associée à la question 1.d, pourquoi celle associée à `biggest_k<<<1, 32>>>(In, Out, 64);` ou à l'une des questions 1.a, 1.b, 1.c n'utilise pas obligatoirement `__syncthreads();` ? (1 point)
3. En se basant sur la question 1.d, définir `biggest_k` permettant de trouver le maximum du tableau `In` lorsqu'on exécute `biggest_k<<<1, 64>>>(In, Out, 121);` (1 point)
4. `atomicMax(int* address, int val);` est une fonction atomic qui permet de faire le maximum entre `val` et `address[0]` puis retourne le résultat dans `address`. En se basant sur la question 1.d, définir `biggest_k` permettant de trouver le maximum du tableau `In` lorsqu'on exécute
 - a) `biggest_k<<<2, 64>>>(In, Out, 256);` (1,5 points)
 - b) `biggest_k<<<(N+127)/128, 128>>>(In, Out, N);` avec $N > 256$ (1 point)